

使用 Python 進行 InnerLoop 開發

來源：<https://codelabs.developers.google.com/innerloop-dev-python?hl=zh-tw>

步驟數：6

瞭解專為在容器化環境中開發 Java 應用程式的軟體工程師所設計的功能和功能，簡化開發工作流程。

目錄

1. [1. 總覽](#)
2. [2. 設定和需求](#)
3. [3. 建立新的 Python 啟動條件應用程式](#)
4. [4. 逐步完成開發程序](#)
5. [5. 開發簡易的 CRUD REST 服務](#)
6. [6. 清除所用資源](#)

步驟 1 · 約 0 分鐘

1. 總覽

本實驗室將示範各項功能和工具，協助軟體工程師在容器化環境中開發 Python 應用程式，簡化開發工作流程。一般容器開發作業需要使用者瞭解容器的詳細資料和容器建構程序。此外，開發人員通常必須中斷流程，離開 IDE 在遠端環境中測試及偵錯應用程式。有了本教學課程中提及的工具和技術，開發人員就能在 IDE 中有效處理容器化應用程式。

學習目標

在本實驗室中，您將瞭解如何在 GCP 中使用容器進行開發，包括：

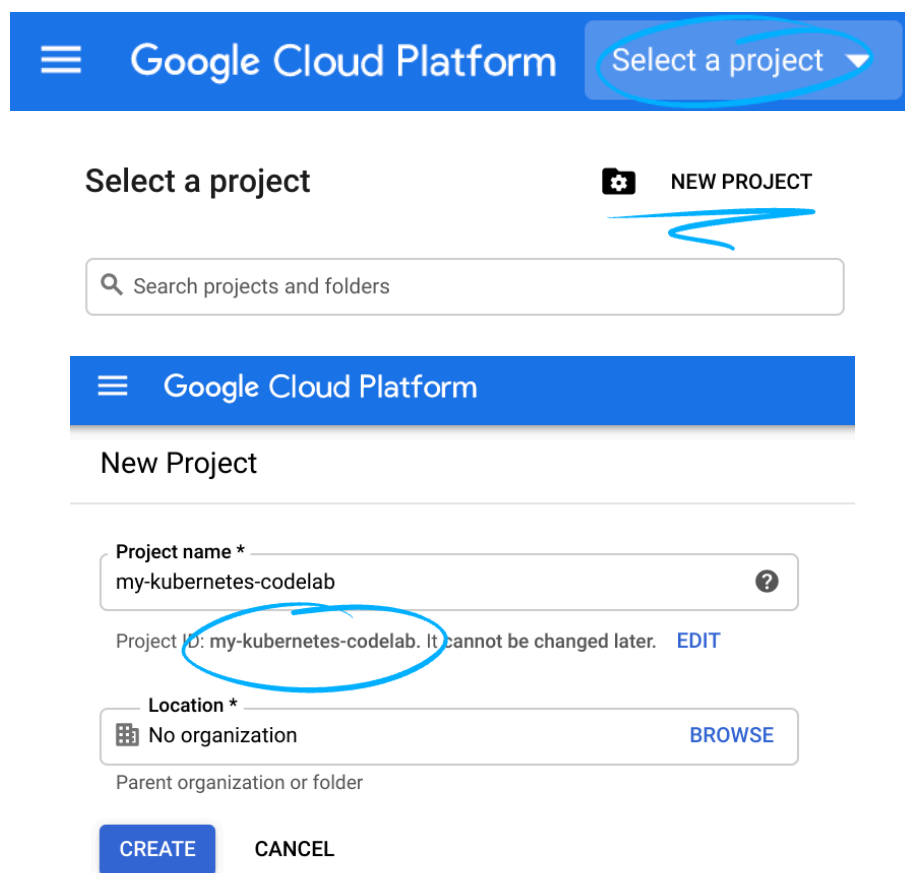
- 建立新的 Python 啟動條件應用程式
 - 逐步完成開發程序
 - 開發簡易的 CRUD REST 服務
-

步驟 2 · 約 2 分鐘

2. 設定和需求

自修實驗室環境設定

1. 登入 [Google Cloud 控制台](#)，然後建立新專案或重複使用現有專案。如果沒有 Gmail 或 Google Workspace 帳戶，請先[建立帳戶](#)。



The screenshot shows the Google Cloud Platform interface. At the top, there is a blue header with the Google Cloud Platform logo and a 'Select a project' dropdown menu, which is circled in blue. Below this, there is a 'Select a project' section with a search bar and a 'NEW PROJECT' button. Below the search bar, there is a 'New Project' section with a 'Project name' field containing 'my-kubernetes-codelab' and a 'Project ID' field containing 'my-kubernetes-codelab', both of which are circled in blue. The 'Project ID' field has a note that it cannot be changed later. Below the 'Project ID' field, there is a 'Location' field set to 'No organization' with a 'BROWSE' button. At the bottom, there are 'CREATE' and 'CANCEL' buttons.

- **專案名稱**是這個專案參與者的顯示名稱。這是 Google API 未使用的字元字串，您隨時可以更新。
- **專案 ID** 在所有 Google Cloud 專案中不得重複，且設定後即無法變更。Cloud 控制台會自動產生專屬字串，通常您不需要在意該字串為何。在大多數程式碼研究室中，您需要參照專案 ID (通常會標示為 `PROJECT_ID`)，因此如果您不喜歡該字串，可以產生另一個隨機字串，或是嘗試使用自己的字串，看看是否可用。專案建立後，系統就會「凍結」該值。
- 還有第三個值，也就是部分 API 使用的「專案編號」。如要進一步瞭解這三種值，請參閱[說明文件](#)。

注意：專案 ID 在全域中不得重複，選定後即為專屬 ID，其他使用者無法使用。您是該 ID 的唯一使用者。專案刪除後，該 ID 就無法再使用。

注意：如果您使用 Gmail 帳戶，可以將預設位置設為「沒有機構」。如果您使用 Google Workspace 帳戶，請選擇適合貴機構的位置。

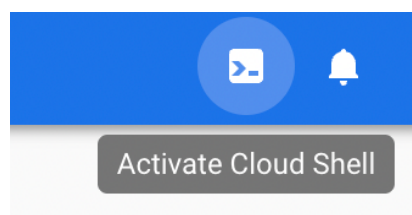
2. 接著，您需要在 Cloud 控制台中[啟用帳單](#)，才能使用 Cloud 資源/API。完成本程式碼研究室的費用應該不高，甚至完全免費。如要停用資源，避免在本教學課程結束後繼續產生帳單費用，請按照程式碼研究室結尾的「清除」操作說明

操作。Google Cloud 新使用者可參加[價值\\$300 美元的免費試用計畫](#)。

啟動 Cloud Shell 編輯器

本實驗室專為 Google Cloud Shell 編輯器設計及測試，如要存取編輯器，請按照下列步驟操作：

1. 前往 <https://console.cloud.google.com> 存取 Google 專案。
2. 按一下右上角的 Cloud Shell 編輯器圖示



3. 視窗底部會開啟新窗格
4. 按一下「開啟編輯器」按鈕



5. 編輯器會開啟，右側顯示檔案總管，中央區域則顯示編輯器
6. 畫面底部也應顯示終端機窗格
7. 如果終端機尚未開啟，請使用 `ctrl+`` 鍵組合開啟新的終端機視窗

環境設定

在 Cloud Shell 設定專案的專案 ID 和專案編號，分別儲存為 `PROJECT_ID` 和 `PROJECT_ID` 變數。

```
export PROJECT_ID=$(gcloud config get-value project)
export PROJECT_NUMBER=$(gcloud projects describe $PROJECT_ID \
  --format='value(projectNumber)')
```

取得原始碼

1. 本實驗室的原始碼位於 GitHub 的 GoogleCloudPlatform 容器開發人員研討會。使用下列指令複製，然後切換至該目錄。

```
git clone https://github.com/GoogleCloudPlatform/container-developer-workshop.git &&
cd container-developer-workshop/labs/python
mkdir music-service && cd music-service
cloudshell workspace .
```

如果終端機尚未開啟，請使用 `ctrl+`` 鍵組合開啟新的終端機視窗

佈建本實驗室使用的基礎架構

在本實驗室中，您會將程式碼部署至 GKE，並存取儲存在 Spanner 資料庫中的資料。下方的設定指令碼會為您準備好基礎架構。佈建程序需要 10 分鐘以上。設定程序處理期間，你可以繼續進行後續步驟。

```
../setup.sh
```

步驟 3 · 約 0 分鐘

3. 建立新的 Python 啟動條件應用程式

1. 建立名為 `requirements.txt` 的檔案，並將下列內容複製到檔案中

```
Flask
gunicorn
google-cloud-spanner
ptvsd==4.3.2
```

2. 建立名為 `app.py` 的檔案，然後將下列程式碼貼入該檔案：

```
import os
from flask import Flask, request, jsonify
from google.cloud import spanner

app = Flask(__name__)

@app.route("/")
def hello_world():
    message="Hello, World!"
    return message

if __name__ == '__main__':
    server_port = os.environ.get('PORT', '8080')
    app.run(debug=False, port=server_port, host='0.0.0.0')
```

3. 建立名為 `Dockerfile` 的檔案，然後將下列內容貼入其中

```
FROM python:3.8
ARG FLASK_DEBUG=0
ENV FLASK_DEBUG=$FLASK_DEBUG
ENV FLASK_APP=app.py
WORKDIR /app
COPY requirements.txt .
RUN pip install --trusted-host pypi.python.org -r requirements.txt
COPY . .
ENTRYPOINT ["python3", "-m", "flask", "run", "--port=8080", "--host=0.0.0.0"]
```

附註：FLASK_DEBUG=1 可讓您自動將程式碼變更重新載入 Python Flask 應用程式。這個 Dockerfile 可讓您將這個值做為建構引數傳遞。

產生資訊清單

在終端機中執行下列指令，產生預設的 `skaffold.yaml` 和 `deployment.yaml`

1. 使用下列指令初始化 Skaffold

```
scaffold init --generate-manifests
```

系統提示時，請使用箭頭鍵移動游標，並按空格鍵選取選項。

您可以選擇：

- 8080，適用於連接埠
- y 儲存設定

更新 Scaffold 設定

- 變更預設應用程式名稱
- 開啟「`scaffold.yaml`」
- 選取目前設為 `dockerfile-image` 的圖片名稱
- 按一下滑鼠右鍵，然後選擇「變更所有出現位置」
- 輸入新名稱，如 `python-app`
- 進一步編輯建構部分，以
- 將 `docker.buildArgs` 新增至票證 `FLASK_DEBUG=1`
- 同步設定，將 IDE 中對 `*.py` 檔案所做的任何變更載入至執行中的容器

編輯後，`scaffold.yaml` 檔案中的建構部分會如下所示：

```
build:
  artifacts:
    - image: python-app
  docker:
    buildArgs:
      FLASK_DEBUG: 1
    dockerfile: Dockerfile
  sync:
    infer:
      - '**/*.py'
```

修改 Kubernetes 設定檔

1. 變更預設名稱

- 開啟 `deployment.yaml` 檔案
 - 選取目前設為 `dockerfile-image` 的圖片名稱
 - 按一下滑鼠右鍵，然後選擇「變更所有出現位置」
 - 輸入新名稱，如 `python-app`
-

步驟 4 · 約 0 分鐘

4. 逐步完成開發程序

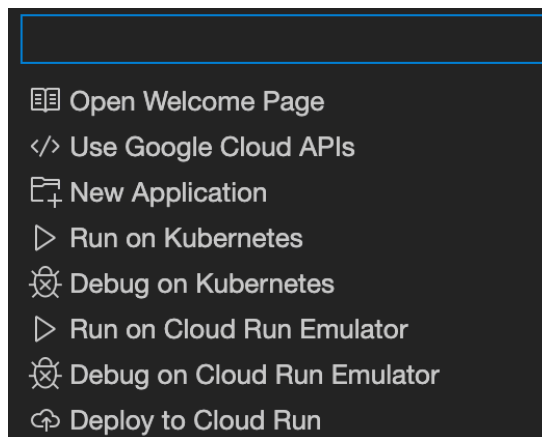
新增商業邏輯後，您現在可以部署及測試應用程式。下一節將重點介紹 Cloud Code 外掛程式的使用方式。這個外掛程式會與 skaffold 整合，簡化開發程序。在後續步驟中部署至 GKE 時，Cloud Code 和 Skaffold 會自動建構容器映像檔、將其推送至 Container Registry，然後將應用程式部署至 GKE。這項作業會在幕後進行，將詳細資料從開發人員流程中抽象化。

部署到 Kubernetes

1. 在 Cloud Shell 編輯器底部的窗格中，選取 Cloud Code 

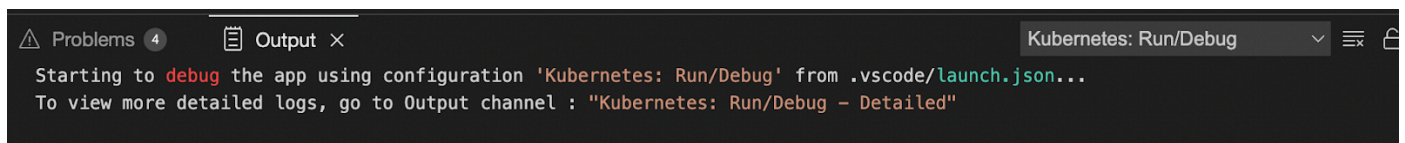
 Cloud Code

2. 在頂端顯示的面板中，選取「Run on Kubernetes」(在 Kubernetes 中執行)。如果系統顯示提示，請選取「Yes」使用目前的 Kubernetes 環境。



這個指令會啟動原始碼的建構作業，然後執行測試。建構和測試程序需要幾分鐘才能完成。這些測試包括單元測試，以及檢查部署環境所設規則的驗證步驟。這個驗證步驟已設定完成，可確保您在開發環境中作業時，也能收到部署問題的警告。

3. 首次執行指令時，畫面頂端會顯示提示，詢問您是否要使用目前的 Kubernetes 內容，請選取「Yes」接受並使用目前的內容。
4. 接著系統會顯示提示，詢問要使用哪個容器登錄服務。按下 Enter 鍵接受預設值
5. 選取下方窗格中的「輸出」分頁標籤，即可查看進度和通知



6. 在右側的管道下拉式選單中選取「Kubernetes: Run/Debug - Detailed」，即可查看其他詳細資料和容器的即時記錄檔串流



建構和測試完成後，「輸出」分頁會顯示 Attached debugger to container "python-app-8476f4bbc-h6ds1" successfully.，並列出網址 <http://localhost:8080>。

7. 在 Cloud Code 終端機中，將游標懸停在輸出內容中的第一個網址 (<http://localhost:8080>) 上，然後在顯示的工具提示中選取「開啟網頁預覽」。
8. 系統會開啟新的瀏覽器分頁，並顯示「Hello, World!」訊息

熱重載

1. 開啟 `app.py` 檔案。
2. 將問候語訊息變更為「Hello from Python」

請注意，在 `Output` 視窗的 `Kubernetes: Run/Debug` 檢視畫面中，監控程式會將更新後的檔案與 `Kubernetes` 中的容器同步

```
Update initiated
Build started for artifact python-app
Build completed for artifact python-app

Deploy started
Deploy completed

Status check started
Resource pod/python-app-6f646ffcb-bb-tn7qd status updated to In Progress
Resource deployment/python-app status updated to In Progress
Resource deployment/python-app status completed successfully
Status check succeeded
...
```

3. 切換至 `Kubernetes: Run/Debug - Detailed` 檢視畫面後，您會發現系統會辨識檔案變更，然後建構並重新部署應用程式

```
files modified: [app.py]
Syncing 1 files for gcr.io/veer-pylab-01/python-app:3c04f58-
dirty@sha256:a42ca7250851c2f2570ff05209f108c5491d13d2b453bb9608c7b4af511109bd
Copying files:map[app.py:[/app/app.py]]to gcr.io/veer-pylab-01/python-app:3c04f58-
dirty@sha256:a42ca7250851c2f2570ff05209f108c5491d13d2b453bb9608c7b4af511109bd
Watching for changes...
[python-app] * Detected change in '/app/app.py', reloading
[python-app] * Restarting with stat
[python-app] * Debugger is active!
[python-app] * Debugger PIN: 744-729-662
```

4. 重新整理瀏覽器即可查看更新後的結果。

偵錯

1. 前往「偵錯」檢視畫面，然後停止目前的執行緒



。

2. 按一下底部選單中的 `Cloud Code`，然後選取 `Debug on Kubernetes`，即可在 `debug` 模式下執行應用程式。
- 在 `Output` 視窗的 `Kubernetes Run/Debug - Detailed` 檢視畫面中，請注意 `scaffold` 會以偵錯模式部署這個應用程式。
3. 首次執行時，系統會提示您容器內的來源位置。這個值與 `Dockerfile` 中的目錄有關。

按下 `Enter` 鍵接受預設值

```
/app

To debug "python-app", confirm or enter the directory in the remote container where the program
can be found. Alternatively, press ESC to skip debugging this container. (Press 'Enter' to confirm or
'Escape' to cancel)
```

應用程式會在幾分鐘內建構及部署完畢。

4. 程序完成時。您會發現偵錯工具已附加。

```
Port forwarding pod/python-app-8bd64cf8b-cskfl in namespace default, remote port 5678
-> http://127.0.0.1:5678
```

5. 底部的狀態列會從藍色變成橘色，表示目前處於偵錯模式。
6. 在 `Kubernetes Run/Debug` 檢視畫面中，請注意已啟動可偵錯的容器

```
*****URLs*****
Forwarded URL from service python-app: http://localhost:8080
Debuggable container started pod/python-app-8bd64cf8b-cskfl:python-app (default)
Update succeeded
*****
```

運用中斷點

1. 開啟 `app.py` 檔案。
2. 找出顯示 `return message` 的陳述式
3. 點選行號左側的空白處，在該行新增中斷點。系統會顯示紅色指標，表示已設定中斷點
4. 重新載入瀏覽器，請注意偵錯工具會在該中斷點停止程序，並允許您調查在 GKE 中遠端執行的應用程式變數和狀態
5. 按一下「變數」部分
6. 按一下「Locals」，即可找到 `"message"` 變數。
7. 按兩下變數名稱「`message`」，然後在彈出式視窗中將值變更為其他內容，例如 `"Greetings from Python"`
8. 按一下偵錯控制面板中的「繼續」按鈕



9. 在瀏覽器中查看回應，現在應該會顯示您剛輸入的更新值。
10. 按下停止按鈕



即可停止「偵錯」模式，再次點選中斷點即可移除。

步驟 5 · 約 0 分鐘

5. 開發簡易的 CRUD REST 服務

此時，您的應用程式已完全設定為容器化開發，且您已透過 Cloud Code 逐步瞭解基本開發工作流程。在接下來的章節中，您將新增 REST 服務端點，連線至 Google Cloud 中的代管資料庫，藉此練習所學內容。

編寫 REST 服務的程式碼

下方程式碼會建立簡單的 REST 服務，並使用 Spanner 做為應用程式的資料庫。將下列程式碼複製到應用程式中，即可建立應用程式。

1. 將 `app.py` 替換為下列內容，建立主要應用程式

```

import os
from flask import Flask, request, jsonify
from google.cloud import spanner

app = Flask(__name__)

instance_id = "music-catalog"

database_id = "musicians"

spanner_client = spanner.Client()
instance = spanner_client.instance(instance_id)
database = instance.database(database_id)

@app.route("/")
def hello_world():
    return "<p>Hello, World!</p>"

@app.route('/singer', methods=['POST'])
def create():
    try:
        request_json = request.get_json()
        singer_id = request_json['singer_id']
        first_name = request_json['first_name']
        last_name = request_json['last_name']
        def insert_singers(transaction):
            row_ct = transaction.execute_update(
                f"INSERT Singers (SingerId, FirstName, LastName) VALUES" \
                f"({singer_id}, '{first_name}', '{last_name}')"
            )
            print("{} record(s) inserted.".format(row_ct))

        database.run_in_transaction(insert_singers)

        return {"Success": True}, 200
    except Exception as e:
        return e

@app.route('/singer', methods=['GET'])
def get_singer():
    try:
        singer_id = request.args.get('singer_id')
        def get_singer():
            first_name = ''
            last_name = ''
            with database.snapshot() as snapshot:

```

```

        results = snapshot.execute_sql(
            f"SELECT SingerId, FirstName, LastName FROM Singers " \
            f"where SingerId = {singer_id}",
        )
        for row in results:
            first_name = row[1]
            last_name = row[2]
            return (first_name, last_name )
        first_name, last_name = get_singer()
        return {"first_name": first_name, "last_name": last_name }, 200
    except Exception as e:
        return e

```

```
@app.route('/singer', methods=['PUT'])
```

```
def update_singer_first_name():
```

```
    try:
```

```
        singer_id = request.args.get('singer_id')
```

```
        request_json = request.get_json()
```

```
        first_name = request_json['first_name']
```

```
    def update_singer(transaction):
```

```
        row_ct = transaction.execute_update(
```

```
            f"UPDATE Singers SET FirstName = '{first_name}' WHERE SingerId = {singer_id}"
```

```
        )
```

```
        print("{} record(s) updated.".format(row_ct))
```

```
    database.run_in_transaction(update_singer)
```

```
    return {"Success": True}, 200
```

```
except Exception as e:
```

```
    return e

```

```
@app.route('/singer', methods=['DELETE'])
```

```
def delete_singer():
```

```
    try:
```

```
        singer_id = request.args.get('singer')
```

```
    def delete_singer(transaction):
```

```
        row_ct = transaction.execute_update(
```

```
            f"DELETE FROM Singers WHERE SingerId = {singer_id}"
```

```
        )
```

```
        print("{} record(s) deleted.".format(row_ct))
```

```
    database.run_in_transaction(delete_singer)
```

```
    return {"Success": True}, 200
```

```
except Exception as e:
```

```
    return e

```

```
port = int(os.environ.get('PORT', 8080))
```

```
if __name__ == '__main__':
    app.run(threaded=True, host='0.0.0.0', port=port)
```

新增資料庫設定

如要安全地連線至 Spanner，請設定應用程式以使用 Workload Identity。這樣一來，應用程式就能以自己的服務帳戶身分運作，並在存取資料庫時擁有個別權限。

1. 更新「`deployment.yaml`」。在檔案結尾新增下列程式碼 (請務必保留以下範例中的 Tab 縮排)

```
serviceAccountName: python-ksa
nodeSelector:
  iam.gke.io/gke-metadata-server-enabled: "true"
```

部署及驗證應用程式

1. 在 Cloud Shell 編輯器底部的窗格中，選取 `Cloud Code`，然後選取畫面頂端的 `Debug on Kubernetes`。
2. 建構和測試完成後，「輸出」分頁會顯示 `Resource deployment/python-app status completed successfully`，並列出網址：「Forwarded URL from service python-app: `http://localhost: 8080`」(從服務 `python-app` 轉送的網址：`http://localhost:8080`)。
3. 新增幾個項目。

在 Cloud Shell 終端機執行下列指令

```
curl -X POST http://localhost:8080/singer -H 'Content-Type: application/json' -d
'{"first_name": "Cat", "last_name": "Meow", "singer_id": 6}'
```

4. 在終端機中執行下列指令，測試 GET

```
curl -X GET http://localhost:8080/singer?singer_id=6
```

5. 測試刪除：現在請執行下列指令，嘗試刪除項目。視需要變更 `item-id` 的值。

```
curl -X DELETE http://localhost:8080/singer?singer_id=6
```

This throws an error message

500 Internal Server Error

找出並修正問題

1. 偵錯模式並找出問題。以下提供幾項訣竅：
 - 我們發現 DELETE 有問題，因為它未傳回預期結果。因此您會在 `delete_singer` 方法的 `app.py` 中設定中斷點。
 - 逐步執行並觀察每個步驟的變數，即可在左側視窗中查看本機變數的值。
 - 如要觀察特定值 (例如 `singer_id` 和 `request.args`)，請將這些變數新增至「監看」視窗。

2. 請注意，指派給 `singer_id` 的值是 `None`。變更程式碼以修正問題。

修正後的程式碼片段如下所示。

```
@app.route('/delete-singer', methods=['DELETE', 'GET'])
def delete_singer():
    try:
        singer_id = request.args.get('singer_id')
```

3. 重新啟動應用程式後，請再次嘗試刪除，確認問題是否解決。
4. 按一下偵錯工具列中的紅色方塊



，停止偵錯工作階段。

6. 清除所用資源

恭喜！在本實驗室中，您從頭開始建立新的 Python 應用程式，並設定該應用程式，使其能有效與容器搭配運作。然後，您按照傳統應用程式堆疊中的相同開發人員流程，將應用程式部署至遠端 GKE 叢集並進行偵錯。

完成實驗室後，請執行下列清理作業：

1. 刪除實驗室中使用的檔案

```
cd ~ && rm -rf container-developer-workshop
```

2. 刪除專案，移除所有相關基礎架構和資源
-